

K-means from Scratch on UNSW-NB15: Euclidean vs. Damped-Diagonal Mahalanobis

Scott Happy (sdh8796)
Rochester Institute of Technology

Abstract—We implement K-means (Lloyd’s algorithm) from scratch and apply it to the UNSW-NB15 network-intrusion dataset, comparing a Euclidean objective against a damped-diagonal Mahalanobis objective. On a class-balanced, z-standardized subsample we select $k = 10$ by elbow and silhouette, evaluate cluster–class agreement in-sample via majority-vote accuracy, precision, recall and F1 plus internal silhouette/V-measure, and compare both runs against a scikit-learn reference. The two geometries produce near-identical partitions; per-feature damping removes heavy-tailed influence without changing the label-based outcome, and we argue that any within-geometry silhouette gain is tautological.

I. Problem & Dataset

UNSW-NB15 (Moustafa & Slay, 2015) is a dataset for classification of network intrusions. It contains a benign class and nine attack classes. The dataset has 175,341 rows and 45 columns; we cluster on the 39 numeric feature columns, dropping the row identifier (id), the binary benign/attack label, the multi-class target attack_cat, and three categorical string fields (proto, service, state). The class label (attack_cat) is used only for the class-balanced sampling design and for evaluation; it is never used as a clustering feature.

For tractability we draw a class-balanced subsample of 3,730 rows (400 per class, except 130 Worms) and z-standardize the features; this does not preserve the natural traffic prevalence. This becomes relevant in later discussions of the simplified Mahalanobis distance, and means our evaluations must be read in the context of the sample size and the data-processing choices. In addition to the base project, we apply damping during the simplified Mahalanobis distance calculation, discussed below. Results can be reproduced with the make reproduce command.

II. Implementation

KMeansScratch implements Lloyd’s algorithm with no library K-means call (the no_library_kmeans_in_scratch check is clean):

- Initialization (`_init_centroids`): random draws k distinct rows; `kmeans++` seeds the first centroid uniformly, then each next centroid with probability proportional to its squared distance to the nearest chosen centroid, with a uniform fallback if every point coincides with a chosen centroid (zero total D^2). Both use the project distance so the seeding matches the objective.

- Assignment (`_assign`): each point to its nearest centroid (argmin of the (n, k) squared-distance matrix).
- Update (`_update`): each centroid moves to the mean of its members. Empty clusters are kept in place (their old centroid is retained) rather than re-seeded: this avoids consuming the RNG stream mid-loop (which would break reproducibility of the restart budget) and did not occur in our converged runs.
- Convergence (`fit`): the loop stops when the Frobenius (L2) norm of the centroid shift is $< \text{tol}$ (10^{-4}) or at `max_iter` (300). scikit-learn’s KMeans also uses an absolute centroid-shift test, but it aggregates the shift as the sum of squared per-centroid changes and compares that sum to `tol`, whereas we use the L2 norm of the full shift matrix. The two are closely related but not identical, so the implementations can settle at marginally different local optima (a small, explainable source of the reference residual in Section V). An absolute shift test is well scaled here because the z-standardized features are unit-variance, so centroid magnitudes are $O(1)$.
- Restarts: $n_{\text{init}} = 50$ restarts; the restart with the lowest inertia is kept (raised from 10 to 50 to improve reference agreement on this high-dimensional, multi-modal dataset; see Section V).

The scikit-learn KMeansReference is used only as an oracle (`compare_to_reference: true`); the submission runs on scratch.

III. Choosing k

A. Euclidean

Over $2 \leq k \leq 15$, $k = 10$ was selected as optimal. The elbow test shows inertia following a relatively smooth curve with an inflection around $8 \leq k \leq 11$; the silhouette score shows a clear peak at $k = 10$. Ten is a reasonable point under both tests, and it matches the ten true classes.

B. Mahalanobis

The selected k was also 10. The elbow curve is more jagged, with a sharp drop then flattening at $k = 10$; the silhouette score again peaks at $k = 10$.

IV. Evaluation

To go from clusters to classifications we use majority vote: every point in a cluster is assigned the majority true class within that cluster.

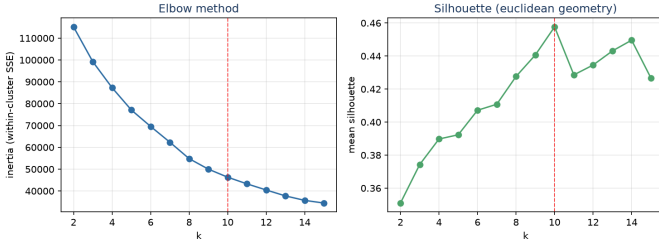


Fig. 1. Euclidean k-selection: inertia (elbow) and silhouette across k .

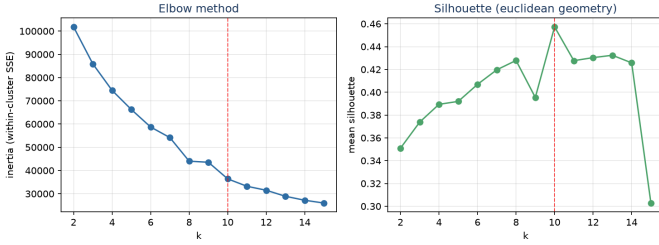


Fig. 2. Mahalanobis k-selection: inertia (elbow) and silhouette across k .

Evaluations were performed only on the data used to fit the model, which limits what we can say about generalization. We nonetheless work with what we have, and additionally inspect principal-component projections to see how tightly the clusters appear. These visual checks should be taken with a grain of salt, but they support the quantitative results.

A. Similarity between metrics

In general the Euclidean and Mahalanobis distances behaved very similarly. As examples, both had accuracy exactly 0.3271, and the silhouette scores (not adjusted for the distance metric used) were 0.4577 for Euclidean and 0.4573 for Mahalanobis. When scored in each run’s own (adjusted) geometry, the Mahalanobis silhouette rises to 0.4832, consistent with the feature scaling discussed in Section V.

As a side note, the only notable difference between the two metrics is inertia, which should not be compared across distance metrics.

The small difference between the two metrics can mean a couple of things:

- The data is largely normalized, so metrics that account for the high variance of certain dimensions have a small effect.
- The data is noisy and the clusters happen to sit in shapes that do not affect the partition. For example, a high-variance, unnormalized dimension far from the other clusters would still cluster fine even if it carried a large share of the inertia.

We find that most features lie in relatively similar ranges, but a few are much larger; this likely drives the higher

within-geometry silhouette under Mahalanobis. Euclidean distance is the primary criterion: we rank the two runs by mean silhouette in the common (standardized) Euclidean space, using V-measure and macro-F1 as label-based cross-checks, and we never compare inertia across the two distance metrics.

B. Agreement with sklearn

Compared against the scikit-learn clusters, our Euclidean ARI is 0.90 and our damped-Mahalanobis ARI is 1.0. Together these indicate a well-aligned reference for the Euclidean run and a clean implementation check for the Mahalanobis run. The ARI of 1.0 for the Mahalanobis run is explained by the reference being fed the same geometry: scikit-learn’s KMeans is Euclidean-only, so we pre-scale each feature by $\sqrt{c_f}$, which makes its Euclidean objective identical to our damped-diagonal-Mahalanobis objective; both then minimize the same function and land in the same optimum. The ARI of 0.90 for the Euclidean run, where the reference is a genuinely independent implementation on the same data, shows high agreement, with the small residual attributable to the different centroid-shift convergence test. This is a good indicator of the in-sample correctness of the implementation.

C. Metrics

The model’s accuracy is far above random, which is good. When choosing between macro and micro precision/recall we consider our stratification: most classes are well balanced, with a few rare attack classes, so we report macro figures to weight the rare classes more heavily.

Macro precision and recall are very similar across the two metrics; for Euclidean they are 0.3560 and 0.3107 respectively. Recall is the more important of the two here, because it indicates the clustering is under-sensitive. The associated macro F1 is 0.2522, which is also quite low. The principal-component view below shows how heavily the feature groups overlap, which would sensibly hurt performance.

D. Class details

The confusion matrix shows predictions concentrated in four classes — Backdoor (45.6%), Fuzzers (33.2%), Generic (10.5%) and Normal (7.5%), which together account for about 97% of all predictions. A further three classes (DoS, Exploits, Worms) are predicted only sparsely, and the remaining three (Analysis, Reconnaissance, Shellcode) are never predicted at all. We look in Euclidean space, because, as noted, the results across the two metrics are extremely similar. For demonstration we show both cluster projections side by side; each displays the data on the two most significant principal components, so the full high-dimensional structure is not represented. Some clusters are highly distinguished, but there are clear regions of confusion, with several groups of several centroids. The higher dimensionality leaves room for sharper

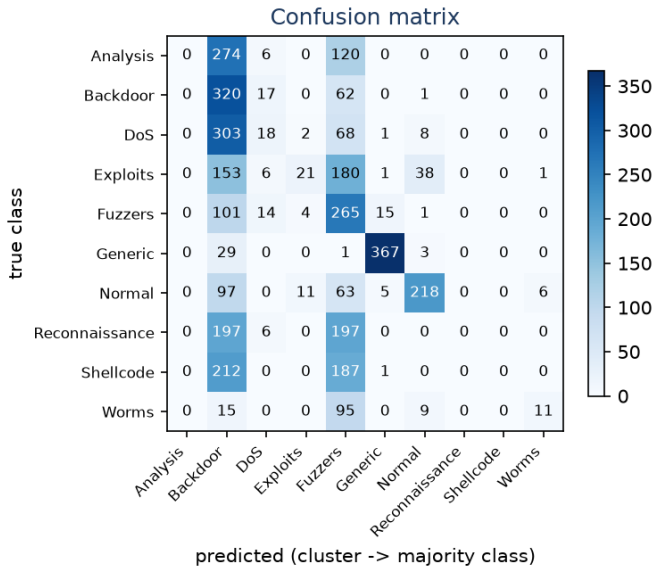


Fig. 3. Euclidean majority-vote confusion matrix.

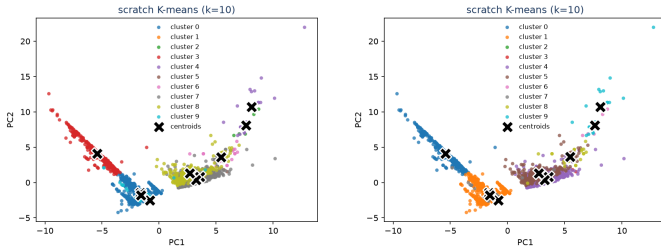


Fig. 4. Cluster projections: Euclidean (left) and Mahalanobis (right).

distinctions, yet the results show that is not quite what happened. The confusion matrix’s concentration in the four classes above roughly matches the number of centroid groups visible in the projection.

V. Euclidean vs. Mahalanobis

As discussed, there are very few differences in evaluation between the Euclidean and Mahalanobis metrics, possibly because most features are similar in scale to one another. The discussion below holds the sample (3,730 points), k ($= 10$), the random seed, and the restart budget ($n_{\text{init}} = 50$) fixed; only the distance changes. Features are used in their fixed CSV column order (the 39 numeric columns), and the diagonal weights are applied positionally to that order.

Most features were already similar in scale and variance, but a few had large skew and variance, which is a good reason to use a Mahalanobis metric. Applying the Mahalanobis metric effectively reduces the feature variance, lowering the chance that highly variable features dominate the clustering. Our simplified (diagonal) Mahalanobis metric does not, however, account greatly for skew, which motivates the additional per-feature scaling below.

TABLE I

Damping factors c_f and distribution statistics of the six damped features.

Feature	c_f	z -skew	max $ z $
stcpb, dtcpb	0.05	1.2 / 1.2	2.5 / 2.6
sbytes	0.30	27.9	34.0
rate	0.10	3.2	4.9
sload	0.10	5.4	14.0
dload	0.20	8.6	12.4

We introduce an additional level of processing by damping the six bandwidth-dominated features (the byte, rate, and load columns). The table below describes their distributions so the reader can check each one’s scale; c_f is the damping factor applied to each feature.

The factors are a judgment, not a monotonic rule applied to the table’s statistics: each value reflects how strongly that column’s raw magnitude would otherwise dominate the geometry, so their ordering need not follow skew or maximum $|z|$. We report those statistics to document the scale of the damped columns, not because the factors are derived from them.

An important consideration is that, because we scale these features down, we obtain inherently better within-geometry silhouette scores, and some other evaluations will look better. This is not evidence of a better clustering: a K-means partition is compact in its own metric by construction, so a within-geometry silhouette gain is tautological and is not counted as an improvement. We have no held-out data to measure generalization, but the damping is a reasonable way to reduce outlier-driven distortion.

VI. Conclusion

It is important to remember that no non-training data was used for evaluation and only a fraction of the original dataset was used. While the principal-component visualization suggests clear shapes for some clusters, there could be differences unaccounted for that would appear if the rest of the dataset had been included in evaluation or training.